

LABORATORY OBSERVATION/RECORD

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	Task 5
Student Name:	R Varshini
Reg. No.:	A24126552115
Date:	21-09-2026

1. TITLE

Interactive Product Calculator Using HTML, CSS and JavaScript

2. AIM

To create a web page containing a product name, quantity and price, and to calculate the total price dynamically using JavaScript functions and event listeners whenever the quantity or price changes, displaying the result without refreshing the page.

3. OBJECTIVE

The objectives of this experiment are:

- To design a simple web form using HTML input elements.
- To style the page using an external CSS file.
- To access HTML elements using the DOM (`document.getElementById`).
- To write a JavaScript function that performs a calculation.
- To attach event listeners so the page responds to user input.
- To update the page content dynamically without a reload.

4. THEORY

HTML defines the structure of the page; here it provides the input fields for product name, quantity and price, and a `div` that shows the total. **CSS** controls the appearance (fonts, spacing, colour of the total). **JavaScript** adds behaviour to the page.

- **DOM** (Document Object Model) represents the page as objects that JavaScript can read and modify.
- **getElementById()** returns a reference to an element using its id.
- **addEventListener()** registers a function to run when an event occurs on an element.
- **input event** fires every time the value of an input field changes, so the result updates while the user types.
- **parseInt()** converts the text value of an input to an integer; `|| 0` gives 0 when the field is empty or invalid.
- **textContent** sets the text of an element, which updates the displayed total instantly.

Flow: User input → input event → `calculateTotal()` → `total = quantity × price` → page updated

5. ALGORITHM / PROCEDURE

1. Create an HTML file with a heading and input fields for product name, quantity and price.
2. Give the quantity, price and total elements unique ids.
3. Link the external CSS file (pbm.css) and the JavaScript file (pbm.js).
4. In JavaScript, get references to the quantity input, price input and total div.
5. Define the function `calculateTotal()`.
6. Inside the function, read quantity and price and convert them to integers (use 0 if empty).
7. Compute `total = quantity × price`.
8. Display the total in the result div using `textContent`.
9. Add an `input` event listener on quantity and price that calls the function.
10. Call the function once at load to show the initial total.
11. Open the page in a browser and enter different values.
12. Verify the actual output against the expected output for each test case.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Product Calculator</title>
  <link rel="stylesheet" href="pbm.css">
</head>
<body>
  <h2>Product Calculator</h2>

  <label>
    Product Name:
    <input type="text" id="productName" placeholder="Enter product name">
  </label>

  <label>
    Quantity:
    <input type="number" id="quantity" value="1" min="1">
  </label>

  <label>
    Price (per unit):
    <input type="number" id="price" value="0" min="0">
  </label>

  <div id="total">Total Price: 0</div>

  <script src="pbm.js"></script>
</body>
</html>
```

pbm.js

```
// Get references to input fields and result div
const quantityInput = document.getElementById('quantity');
const priceInput = document.getElementById('price');
const totalDiv = document.getElementById('total');

// Function to calculate and display total
function calculateTotal() {
  const quantity = parseInt(quantityInput.value) || 0;
  const price = parseInt(priceInput.value) || 0;
  const total = quantity * price;
  totalDiv.textContent = "Total Price: " + total;
}

// Add event listeners to inputs
quantityInput.addEventListener('input', calculateTotal);
priceInput.addEventListener('input', calculateTotal);

// Initial calculation
calculateTotal();
```

pbm.css

```
body {
  font-family: Arial, sans-serif;
  margin: 40px;
}

label {
  display: block;
  margin-top: 10px;
}

input {
  margin-left: 10px;
}

#total {
  margin-top: 20px;
  font-size: 18px;
  font-weight: bold;
  color: green;
}
```

7. CODE EXPLANATION

Code	Explanation
<code>getElementById('quantity')</code>	Gets the quantity input element from the page.
<code>getElementById('price')</code>	Gets the price input element from the page.
<code>getElementById('total')</code>	Gets the div in which the total price is shown.
<code>calculateTotal()</code>	Reads both inputs, multiplies them and shows the total.
<code>parseInt(...) 0</code>	Converts the text to a number; uses 0 if the field is empty.
<code>textContent</code>	Replaces the text of the total div with the new result.
<code>addEventListener('input', ...)</code>	Runs <code>calculateTotal()</code> every time quantity or price changes.
<code>calculateTotal();</code>	Displays the initial total when the page first loads.
<code><link rel="stylesheet"></code>	Attaches the external CSS file to the page.
<code><script src="pbm.js"></code>	Attaches the JavaScript file at the end of the body.

8. TEST CASES AND EXECUTION DATA

Each test case was executed in the browser by entering the values below and observing the total shown on the page. The results were recorded exactly as displayed.

Test Case	Product	Qty	Price	Expected Result	Actual Result	Status
TC-01	Laptop	2	500	Total Price: 1000	Total Price: 1000	Pass
TC-02	Pen	10	5	Total Price: 50	Total Price: 50	Pass
TC-03	Notebook	1 (default)	0 (default)	Total Price: 0	Total Price: 0	Pass
TC-04a	Mouse	3	250	Total Price: 750	Total Price: 750	Pass
TC-04b	Mouse	3 → 4 (changed)	250	Total updates to 1000 without reload	Total Price: 1000 (no reload)	Pass
TC-05	Keyboard	2	Left empty	Total Price: 0 (empty treated as 0)	Total Price: 0	Pass

9. OUTPUT

Output 1: Initial page load

Product Calculator
Product Name:
Quantity:
Price (per unit):
Total Price: 0

On load the total is calculated once and shows **Total Price: 0** (quantity 1 × price 0).

Output 2: TC-01 – Laptop, Quantity 2, Price 500

Product Calculator
Product Name:
Quantity:
Price (per unit):
Total Price: 1000

Total Price: 1000 displayed.

Output 3: TC-02 – Pen, Quantity 10, Price 5

Product Calculator
Product Name:
Quantity:
Price (per unit):
Total Price: 50

Total Price: 50 displayed.

Output 4: TC-03 – Notebook, default quantity and price

Product Calculator
Product Name:
Quantity:
Price (per unit):
Total Price: 0

Total Price: 0 displayed.

Output 5: TC-04a – Mouse, Quantity 3, Price 250

Product Calculator
Product Name:
Quantity:
Price (per unit):
Total Price: 750

Total Price: 750 displayed.

Output 6: TC-04b – Quantity changed from 3 to 4 (same page, no refresh)

Product Calculator
Product Name:
Quantity:
Price (per unit):
Total Price: 1000

The total updated instantly to 1000 as soon as the quantity changed.

Output 7: TC-05 – Keyboard, Quantity 2, Price left empty

Product Calculator
Product Name:
Quantity:
Price (per unit):
Total Price: 0

Total Price: 0 displayed; the empty price is treated as 0 and no error occurs.

10. RESULT

Thus, an interactive Product Calculator web page was successfully developed using HTML, CSS and JavaScript. JavaScript functions and event listeners were used to calculate the total price dynamically whenever the quantity or price changed, and the result was displayed without refreshing the page. The application was verified using multiple test cases.